UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

Interface com o Teclado Matricial e Display 16x2

Docente: Prof. Dr. Ruhan Pontes Policarpo de Souza
Discente: Carlos Gabriel Baratieri — 2706598
Discente: Leonardo Higuti Ruzza — 2525470
Discente: Luiz Fernando Coimbra Hintz — 2559145
Turma: C51B

O presente documento tem como objetivo relatar a terceira prática laboratorial da disciplina de Sistemas Microcontrolados, envolvendo interface com o Teclado Matricial e Display 16x2.

Sistemas Microcontrolados — EC45C
22/09/2026

27 de setembro de 2026 — 21:46
Discente: Carlos Gabriel Baratieri
Discente: Luiz F. Coimbra Hintz
Discente: Leonardo Higuti Ruzza

1 Introdução

O presente experimento tem como objetivo abordar uma prática referente ao teclado matricial e o display 16x2 por intermédio de ferramentas, como o Proteus e o compilador MPLAB X IDE, em conjunto com o kit didático XM118 – Exsto, fazendo uso de gravação interna e externa. Em detalhe, como será discorrido nesse relatório, foram usados diversos kits e testes nesses kits, pois muitos estavam com curto. O professor disponibilizou o esquemático que era para ser usado pelo Proteus para que fosse possível simular o firmware. Além disso, gravamos o firmware no kit para testar o código em execução real.

Basicamente, o firmware a ser desenvolvido tinha várias etapas, a primeira era mostrar na linha de cima a data e o código da turma, na segunda linha a lógica era ler o teclado (teclas pressionadas) e apresentar na tela. A lógica do firmware será descrita ao longo do relatório.

2 Materiais e Métodos

Os materiais utilizados para a realização do experimento foram o microcontrolador PIC18F4550, o kit didático XM118 – Exsto (com PIC18F4550, teclado matricial e display 16x2 integrados), um computador com o MPLAB X IDE versão 5.20, o software Proteus para simulação, cabos para conexão e osciloscópio. Na Figura 1, podemos ver uma foto do kit utilizado, mas foram utilizados ao todo 5 kits, pois muitos estavam com curto, mas será explicado no decorrer do relatório.

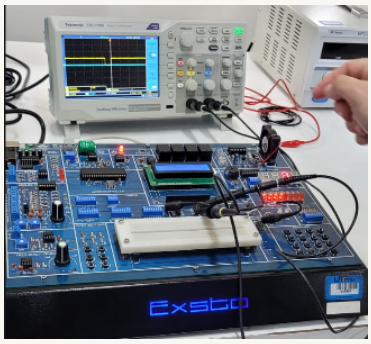


Figura 1: Kit didático utilizado.

Na Figura 2, podemos observar uma amostra do compilador e nesse caso estávamos comentando botões para ver quais estavam com curto.

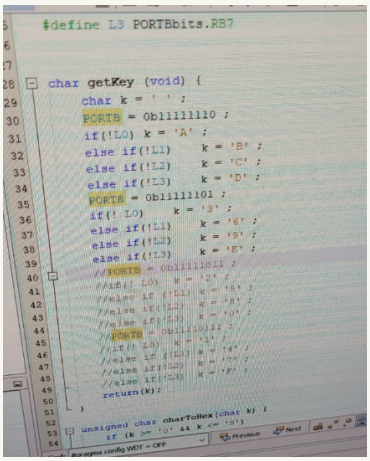


Figura 2: Compilador com botões comentados.

Na Figura 3, podemos ver o osciloscópio que usamos para ver os sinais das teclas, para observar o curto.

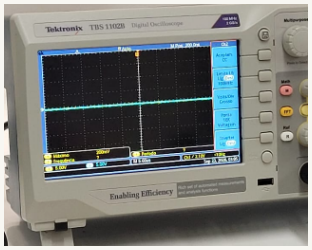


Figura 3: Osciloscópio utilizado.

Abaixo, temos o enunciado. Toda a lógica vai ser discorrido na próxima seção.

> Desenvolva um firmware capaz de realizar a leitura de um teclado matricial hexadecimal (0 a F, 4 linhas e 4 colunas), e apresentar o resultado da leitura sequencialmente no display LCD alfanumérico. O funcionamento deverá seguir os seguintes requisitos e formato:
>
> - Na primeira linha do display, deve-se apresentar a data e o código da turma;
> - Leia as teclas de um teclado matricial (0 a F);
> - Na segunda linha do display, apresente no primeiro endereço (0xC0) a tecla recém-pressionada;
> - Repita o processo de leitura das teclas, e sempre que uma nova tecla for pressionada, o valor correspondente deve ser apresentado no endereço seguinte (0xC1, ...) ao que foi exibido anteriormente, criando um histórico das teclas pressionadas.
> - O processo é repetido indefinidamente, apresentando sempre o conjunto de teclas pressionadas.
> - O processo retorna ao endereço inicial (0xC0) após preenchidos todos os 16 endereços disponíveis.

Como dito anteriormente, foram testados diversos kits, pois a maioria estava com curto nas teclas. Esse defeito pode ser visualizado nos seguintes vídeos, de autoria própria e gravados durante o experimento:

- Vídeo 1: Vídeo 1
- Vídeo 2: Vídeo 2

Foi orientado durante a prática comentar algumas linhas até descobrir onde estava dando defeito, mas isso desencadeou diversas teclas comentadas, alguns kits estavam digitando sozinhos, outros nem funcionavam.

Mas, no Proteus funcionou e no primeiro kit parcialmente, mesmo com o curto. O professor foi fundamental e passou boas práticas do uso do equipamento osciloscópio para averiguar curto. Ele explicou sobre as diferenças de tensão, assim como complementares, como pode ser melhor visualizado na Figura 1.

> **Observação:** Não usar o autotest durante o uso do osciloscópio.

Abaixo, na Figura 4 temos a ordem de manipulação dos espaços do LCD que será usada para futuras explicações.

80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF

Figura 4: LCD alfanumérico *16x2* - Posições de Memória (Presente no Roteiro da Prática)

3 Resultados

Abaixo, temos o código que foi testado tanto no kit quanto no Proteus.

```c
#include <xc.h>
#include "nxlcd.h"

#pragma config PLLDIV = 5
#pragma config WDT = OFF
#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config DEBUG = OFF
#pragma config MCLRE = ON

#define _XTAL_FREQ 20000000

#define C1 PORTBbits.RB0
#define C2 PORTBbits.RB1
#define C3 PORTBbits.RB2
#define C4 PORTBbits.RB3
#define L0 PORTBbits.RB4
#define L1 PORTBbits.RB5
#define L2 PORTBbits.RB6
#define L3 PORTBbits.RB7

char getKey(void) {
    char k = ' ';

    PORTB = 0b11111110;
    if(!L0) k = 'A';
    else if(!L1) k = 'B';
    else if(!L2) k = 'C';
    else if(!L3) k = 'D';

    PORTB = 0b11111101;
    if(!L0) k = '3';
    else if(!L1) k = '6';
    else if(!L2) k = '9';
    else if(!L3) k = 'E';

    PORTB = 0b11111011;
    if(!L0) k = '2';
    else if(!L1) k = '5';
    else if(!L2) k = '8';
    else if(!L3) k = '0';

    PORTB = 0b11110111;
    if(!L0) k = '1';
    else if(!L1) k = '4';
    else if(!L2) k = '7';
    else if(!L3) k = 'F';

    return(k);
}

unsigned char charToHex(char k) {
    if (k >= '0' && k <= '9')
        return k - '0';
    else if (k >= 'A' && k <= 'F')
        return k - 'A' + 10;
    else
        return 0xFF;
}

void main(void) {
    char tecla = ' ';
    unsigned char posicao = 0xC0;

    OpenXLCD(FOUR_BIT & LINES_5X7);
    WriteCmdXLCD(0x01);
    __delay_ms(2);
    WriteCmdXLCD(0xC0);
    WriteCmdXLCD(0x80);
    putsXLCD("EC45C - 22/09/26");

    TRISB = 0xF0;
    TRISD = 0x00;

    while (1) {
        tecla = getKey();

        if (tecla != ' ') {
            WriteCmdXLCD(posicao);
            putcXLCD(tecla);
            posicao++;

            if (posicao > 0xCF) {
                posicao = 0xC0;
            }
            __delay_ms(150);
        }
    }
}
```



Primeiramente, há de declarar algo, foram usados alguns arquivos disponibilizados pelo professor, como o arquivo de cabeçalho `nxlcd.h` e o arquivo `*.c` denominado `nxlcd.c`, por simplificação de conceitos, vamos chamar de bibliotecas. Abaixo, temos um exemplo dos arquivos no computador.

```
fuyu@fuyu:~/Downloads/proj2/proj2.X$ ls
build  debug  dist  L3.pdsprj  Makefile  nbproject  newmain.c  nxlcd.c  nxlcd.h
```

A biblioteca foi inserida usando a diretiva de inclusão `include`, sendo incluído um arquivo de cabeçalho, por ser algo local, é usado `"..."`, assim, a etapa de pré-processamento procura no diretório atual, diferentemente de uma biblioteca padrão, em que é usado `<...>`. Assim, temos as bibliotecas abaixo.

```
#include <xc.h>
#include "nxlcd.h"
```

Temos, abaixo, algumas configurações padrão disponibilizadas previamente pelo professor.

```
#pragma config PLLDIV = 5
#pragma config WDT = OFF
#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config DEBUG = OFF
#pragma config MCLRE = ON

#define _XTAL_FREQ 20000000
```

E temos alguns macros, que vamos chamar de apelidos, também disponibilizados pelo professor. Eles são de suma importância para deixar o código mais limpo. Ao invés de inserir uma "string"ou um comando maior como `PORTBbits.RBX`, são definidos apelidos.

```
#define C1 PORTBbits.RB0
#define C2 PORTBbits.RB1
#define C3 PORTBbits.RB2
#define C4 PORTBbits.RB3
#define L0 PORTBbits.RB4
#define L1 PORTBbits.RB5
#define L2 PORTBbits.RB6
#define L3 PORTBbits.RB7
```

Novamente, temos algumas funções que foram disponibilizadas pelo professor. A função `getKey()` basicamente faz o mapeamento do teclado 4x4. Aqui são usados os apelidos definidos anteriormente, em que usamos linhas (`RB4--RB7; L0, L1, L2, L3`) e colunas (`RB0--RB3; C1, C2, C3, C4`). A varredura ativa uma coluna por vez (nível lógico 0) e realiza a leitura das linhas.

```
char getKey (void) {
  char k = ' ' ;
  PORTB = 0b11111110 ;
  if(!L0)  k = 'A' ;
  else if(!L1)  k = 'B';
  else if(!L2)  k = 'C';
  else if(!L3)  k = 'D';
  PORTB = 0b11111101 ;
  if(! L0)  k = '3' ;
  else if(!L1)  k = '6';
  else if(!L2)  k = '9';
  else if(!L3)  k = 'E';
  PORTB = 0b11111011 ;
  if(! L0)  k = '2' ;
  else if (!L1)  k = '5';
  else if(!L2)  k = '8';
  else if(!L3)  k = '0';
  PORTB = 0b11110111 ;
  if(! L0)  k = '1' ;
  else if (!L1)  k = '4';
  else if(!L2)  k = '7';
  else if(!L3)  k = 'F';
  return(k);
}
```

Após ativar a coluna, ele testa as linhas. É usado o operador `!` por conta da lógica interna do kit.

```
0b11111110 -> coluna 1 ativa (RB0 = 0)
0b11111101 -> coluna 2 ativa (RB1 = 0)
0b11111011 -> coluna 3 ativa (RB2 = 0)
0b11110111 -> coluna 4 ativa (RB3 = 0)
```

`char k = ' ';` é o valor inicial, e o `return k` basicamente tem a função de devolver o caractere correspondido pela letra.

Abaixo, temos a Tabela 1 que mostra um pouco do funcionamento lógico do mapeamento do teclado.

Tabela 1: Mapa de teclas formado.

Coluna ativa	L0	L1	L2	L3
C1 (11111110)	A	B	C	D
C2 (11111101)	3	6	9	E
C3 (11111011)	2	5	8	0
C4 (11110111)	1	4	7	F

Outra função fornecida é a `charToHex()`, que basicamente faz a conversão para hexadecimal, pegando como argumento o retorno da função anterior, que é um caractere (dentro do limite de 0 até 9 ou A até F), no seu valor numérico. Não foi usado no desenvolvimento, mas é uma função interessante.

```
unsigned char charToHex(char k) {
    if (k >= '0' && k <= '9')
        return k - '0';
    else if (k >= 'A' && k <= 'F')
        return k - 'A' + 10;
    else
        return 0xFF;
}
```

Na Figura 5 temos uma demonstração de cada condicional que foi desenhada no Excalidraw.

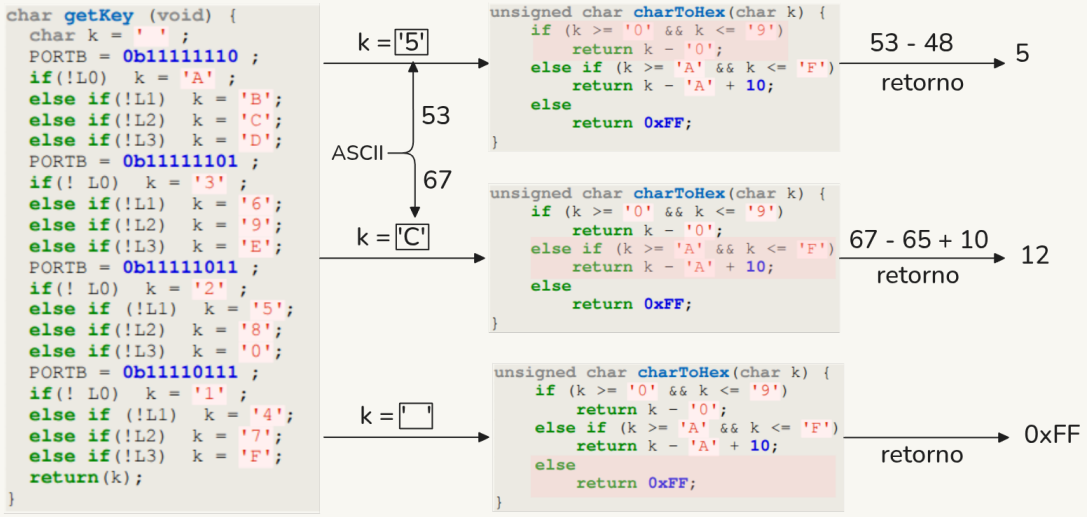


Figura 5: Demonstração das condicionais desenhada no Excalidraw.

Antes da parte lógica do código, será apresentado um trecho retirado dos códigos de exemplo disponibilizados pelo professor, utilizados durante as aulas e explicados no roteiro. A função `OpenXLCD(FOUR_BIT & LINES_5X7)` inicializa o LCD no modo de comunicação de 4 bits, utilizando a configuração de caracteres 5x7. A função `WriteCmdXLCD(0x01)` envia o comando para limpar o LCD, enquanto `__delay_ms(2)` aguarda a execução do comando. Em seguida, `WriteCmdXLCD(0xC0)` indica o endereço inicial da segunda linha do LCD, logo depois `WriteCmdXLCD(0x80)` reposiciona o cursor no início da primeira linha. Por fim, `putsXLCD` permite escrever o texto *"EC45C - 22/09/26"* a partir dessa posição, conforme apresentado anteriormente.

```
    OpenXLCD(FOUR_BIT & LINES_5X7);
    WriteCmdXLCD(0x01);
    __delay_ms(2);
    WriteCmdXLCD(0xC0);
    WriteCmdXLCD(0x80);
    putsXLCD("EC45C - 22/09/26");
```

O código atente os primeiros requisitos do enunciado. Os códigos utilizados foram disponibilizados pelo professor, assim todo crédito intelectual vai para ele ou para a pessoa que escreveu essa porção de código. A disposição das posições de memória do LCD pode ser observada na matriz apresentada na Figura 4.



As demais tarefas consistiam basicamente em pressionar as teclas, fazendo com que os valores correspondentes aparecessem no display. As funções explicadas anteriormente já realizam a leitura e identificação das teclas, restando configurar as entradas e saídas, verificar quais teclas foram pressionadas e escrevê-las na tela. Também foi necessário reajustar a posição de escrita para os novos caracteres. Ao chegar ao final das posições disponíveis, o endereço era realocado para o início novamente.

```
void main(void) {
    char tecla = ' ';
    unsigned char posicao = 0xC0;
    // [...]
    TRISB = 0xF0;
    TRISD = 0x00;
}
```

A variável `tecla` armazena a tecla identificada, enquanto `posicao` indica o endereço em que o próximo caractere será escrito no LCD. A configuração `TRISB = 0xF0` (*11110000*) define os quatro bits menos significativos da porta B como saída e os quatro mais significativos como entrada. O `TRISD = 0x00` configura a porta D como saída.

O comando `while(1)` mantém o programa em execução contínua, permitindo verificar constantemente se alguma tecla foi pressionada.

```
while (1) {
    // [...]
}
```

Dentro do laço, a função `getKey()` é utilizada para verificar qual tecla foi pressionada.

```
while (1) {
    tecla = getKey();
    // [...]
}
```

O comando `if (tecla != ' ')` verifica se uma tecla foi identificada, ou seja, se o valor é diferente de `' '`, que representa nenhuma tecla pressionada.

```
while (1) {
    tecla = getKey();
    if (tecla != ' ') {
        // [...]
    }
}
```

Caso uma tecla seja identificada, a função `WriteCmdXLCD(posicao)` define a posição de memória do LCD em que o caractere será escrito. Em seguida, `putcXLCD(tecla)` escreve no display o caractere armazenado na variável `tecla`. A variável `posicao` é incrementada para que o próximo caractere seja escrito na posição seguinte.

```
while (1) {
    tecla = getKey();
    if (tecla != ' ') {
        WriteCmdXLCD(posicao);
        putcXLCD(tecla);
        posicao++;
        // [...]
    }
}
```

Por fim, quando todas as posições disponíveis da segunda linha forem utilizadas, a variável `posicao` retorna para `0xC0`, permitindo que o processo seja repetido.

```
while (1) {
    tecla = getKey();
    if (tecla != ' ') {
        WriteCmdXLCD(posicao);
        putcXLCD(tecla);
        posicao++;

        if (posicao > 0xCF) {
            posicao = 0xC0;
        }
        __delay_ms(150);
    }
}
```

Abaixo, temos o diagrama usado no Proteus.

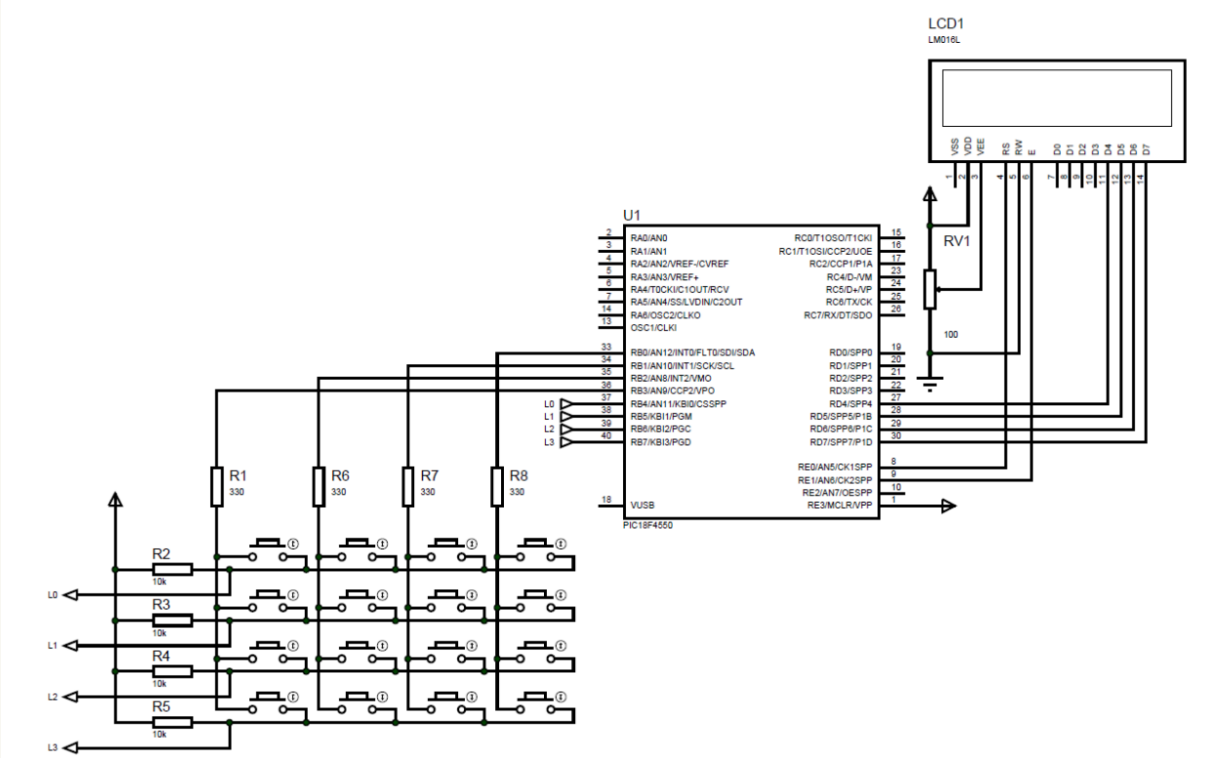


Figura 6: Diagrama no Proteus

4 Conclusão

Neste experimento, foi desenvolvido um firmware para o PIC18F4550, com foco na leitura do teclado matricial e na apresentação das teclas no display LCD 16x2. O firmware foi testado no Proteus e também no kit didático XM118 – Exsto, utilizando o MPLAB X IDE 5.20.

O código funcionou corretamente no Proteus e parcialmente nos kits, devido aos problemas de curto encontrados em algumas teclas. Mesmo sob essas problemáticas, foi possível realizar a leitura das teclas e visualização no LCD. Assim, os objetivos do experimento foram atendidos.